Corrected Gap Analysis and Detailed Implementation of Five Novel Solutions for Few-Shot Object Detection

FSOD Research

March 2026

1 Correction: Seed Evaluation Protocol

1.1 Paper's Actual Protocol (from official `run_voc.sh`)

Examination of the official DeFRCN repository reveals the **exact** VOC FSOD evaluation protocol:

Listing 1: Official DeFRCN VOC FSOD protocol

```
# --> 1. FSRW-like, i.e. run seed0 10 times
#        (the FSOD results on voc in most papers)
for repeat_id in 0 1 2 3 4 5 6 7 8 9
do
    for shot in 1 2 3 5 10
    do
        for seed in 0
        do
            # ... train and evaluate ...
        done
    done
done
```

Conclusion: The paper uses **seed 0 only** for VOC FSOD—the same as our setup. The "averaged over multiple runs" refers to 10 *repeat runs* of the same seed (same support images, different random training initialization), **not** different seed selections. Therefore, single-seed evaluation is **NOT a source of the performance gap**.

1.2 Revised Configuration Comparison

1.3 Revised Root Causes (Corrected)

1. ~~Single seed vs. multi-seed~~ → **Both use seed 0.** Not a factor.

2. **10 repeats vs. 1 repeat** (Revised): The paper averages 10 independent training runs. This reduces variance from stochastic training (random weight init for novel classifier, mini-batch sampling order, dropout). Our single run could be unlucky. *Estimated impact:* ± 1–2 AP_{50}.

3. **Batch size 16 → 8**: Still the dominant factor. Although total image throughput matches, the per-step gradient quality differs. With BS=8, each gradient update is noisier ($\sqrt{2}\times$ higher variance). Crucially, with only 1–5 images per novel class, even the total data is identical across batches—the batch composition is more homogeneous with BS=8 (more

Table 1: Corrected comparison between paper and our setup

Parameter	**Paper (Official)**	**Ours**
GPUs	8	1
Total batch size (IMS_PER_BATCH)	16	8
Images per GPU	2	8
Base Training		
BASE_LR	0.02	0.01 (linearly scaled)
MAX_ITER	15,000	30,000 (2× to match throughput)
STEPS	(10000, 13300)	(20000, 26600) (scaled)
Total images seen	240,000	240,000
Novel 1-shot Fine-tuning		
BASE_LR	0.01	0.005 (linearly scaled)
MAX_ITER	800	1,600 (2× to match throughput)
STEPS	(640,)	(1280,) (scaled)
Total images seen	12,800	12,800
Seed	0	0
Repeats	10 (averaged)	1

repetitions of the same 5 images per epoch), potentially causing different convergence dynamics.

4. **Multi-GPU distributed training effects**: The paper uses 8 GPUs with 2 images each. Detectron2's distributed training performs `allreduce` on gradients across GPUs *before* updating, which averages gradients from 8 different mini-batches. This is equivalent to a single-GPU BS=16 in terms of gradient quality, but the gradient averaging across GPUs provides an implicit regularization effect that single-GPU BS=8 lacks.

5. **Per-GPU BN statistics**: With 8 GPUs and BS=16 total (2 per GPU), each GPU computes BN statistics from only 2 images. Our 1 GPU with BS=8 computes BN from 8 images—*actually better* local BN statistics. However, DeFRCN uses `FrozenBN` (frozen from ImageNet pretraining), so this is likely irrelevant.

6. **Novel methods not executing**: Unchanged from previous analysis. The identical results across contrastive/self-distillation/frequency methods prove the code has integration bugs.

Revised gap attribution: The ∼3.5 AP_{50} gap (50.12 vs. 53.6) is primarily from: (a) BS=8 gradient noise (∼1.5–2.5 AP), (b) single repeat vs. 10-repeat averaging (∼1–2 AP), and (c) non-linear LR scaling effects (∼0.5–1 AP).

2 The Overfitting Problem in FSOD

Before presenting solutions, we must rigorously characterize *why* FSOD is prone to overfitting, as this constrains what solutions are viable.

2.1 Why FSOD Overfits

1. **Extreme data scarcity:** In 1-shot VOC FSOD, we have 5 training images (1 per novel class) containing ∼5–10 object instances total. The classifier head has $5 \times 2048 + 5 = 10,245$ parameters for just the final FC layer—**more parameters than training pixels**.

2. **Large pre-trained backbone:** ResNet-101 has ∼44M parameters. Even with most frozen, the unfrozen layers (classifier, box regressor, portions of res5) have ∼500K–2M learnable parameters vs. 5 training images.

3. **Domain shift:** ImageNet features ≠ VOC detection features. Fine-tuning on 5 images cannot bridge this gap without overfitting to the specific visual patterns of those 5 images.

4. **Class imbalance:** The RPN generates ∼1000 proposals per image. Only a tiny fraction are positives for novel classes. The classifier is overwhelmed by background negatives, learning to suppress all predictions (underfitting novel classes) or memorizing the few positives (overfitting).

2.2 The Overfitting Constraint on Novel Solutions

Any proposed solution must satisfy:

- **Zero or near-zero additional learnable parameters** during novel fine-tuning
- **Inference-time only** modifications are safest (no training overfitting possible)
- If training modifications are needed, they must act on the **base training stage** (abundant data) and **transfer** to novel classes without fine-tuning
- Any learnable component added during novel fine-tuning must have **strong inductive biases** that constrain the solution space

3 Solution 1: Batch-Agnostic Few-Shot Detection (BA-FSD)

3.1 Motivation

The core problem: batch size affects BN statistics during *base training*, and this propagates to novel class performance. While DeFRCN uses FrozenBN (frozen ImageNet statistics), the convergence dynamics during base training with different batch sizes still produce different learned weights. The solution must make training invariant to batch size.

3.2 Detailed Implementation

3.2.1 Step 1: Replace BN with GN+WS in the Backbone

Replace all BatchNorm layers in ResNet-101 with Group Normalization (32 groups) + Weight Standardization:

Listing 2: GN+WS replacement in ResNet backbone

```
import torch.nn as nn
import torch.nn.functional as F

class WeightStandardizedConv2d(nn.Conv2d):
    def forward(self, x):
        weight = self.weight
        # Weight Standardization: normalize kernel weights
        mean = weight.mean(dim=[1, 2, 3], keepdim=True)
        std = weight.std(dim=[1, 2, 3], keepdim=True) + 1e-5
        weight = (weight - mean) / std
        return F.conv2d(x, weight, self.bias, self.stride,
                        self.padding, self.dilation, self.groups)

def convert_bn_to_gn_ws(model, num_groups=32):
    """Replace all BN layers with GN and Conv2d with WS-Conv2d."""
    for name, module in model.named_children():
```

```
        if isinstance(module, (nn.BatchNorm2d, FrozenBatchNorm2d)):
            num_channels = module.num_features
            gn = nn.GroupNorm(min(num_groups, num_channels),
                              num_channels, affine=True)
            setattr(model, name, gn)
        elif isinstance(module, nn.Conv2d) and module.kernel_size !=
            (1,1):
            ws_conv = WeightStandardizedConv2d(
                module.in_channels, module.out_channels,
                module.kernel_size, stride=module.stride,
                padding=module.padding, dilation=module.dilation,
                groups=module.groups, bias=module.bias is not None
            )
            ws_conv.weight.data.copy_(module.weight.data)
            if module.bias is not None:
                ws_conv.bias.data.copy_(module.bias.data)
            setattr(model, name, ws_conv)
        else:
            convert_bn_to_gn_ws(module, num_groups)
```

3.2.2 Step 2: Task-Adaptive Normalization (TAN) for Novel Classes

During novel fine-tuning, learn class-conditioned affine parameters for each GN layer. A small hypernetwork generates (γ_c, β_c) from the support set:

Listing 3: Task-Adaptive Normalization module

```
class TaskAdaptiveNorm(nn.Module):
    def __init__(self, num_channels, support_dim=2048):
        super().__init__()
        self.gn = nn.GroupNorm(32, num_channels, affine=False)
        # Hypernetwork: support features -> affine params
        # Very small: only 2 * num_channels output params
        self.hyper = nn.Sequential(
            nn.AdaptiveAvgPool2d(1),
            nn.Flatten(),
            nn.Linear(support_dim, 64),  # bottleneck
            nn.ReLU(inplace=True),
            nn.Linear(64, 2 * num_channels)  # gamma + beta
        )
        # Initialize to identity transform
        nn.init.zeros_(self.hyper[-1].weight)
        nn.init.zeros_(self.hyper[-1].bias)
        self.hyper[-1].bias.data[:num_channels] = 1.0  # gamma=1

    def forward(self, x, support_feat=None):
        out = self.gn(x)
        if support_feat is not None:
            params = self.hyper(support_feat)
            gamma = params[:, :out.size(1)].unsqueeze(-1).unsqueeze(-1)
            beta = params[:, out.size(1):].unsqueeze(-1).unsqueeze(-1)
            out = gamma * out + beta
        return out
```

3.2.3 Step 3: Integration with DeFRCN Pipeline

1. **Base training:** Train ResNet-101+GN+WS from ImageNet-pretrained weights (re-initialize GN from BN stats). Train with standard DeFRCN pipeline. This base model is now batch-size invariant.
2. **Novel fine-tuning:** Freeze backbone. Only train the classifier head + TAN hypernetworks. The hypernetwork parameters total $\sim 64\times 2048 + 64\times 2\times C$ per layer $\approx$ 130K params, but they are shared across classes and conditioned on the support set, preventing per-class overfitting.
3. **Inference:** Compute TAN affine params once from the K-shot support set, cache them, and apply during inference.

3.3 Why This Avoids Overfitting

- **GN+WS modification is entirely in base training** (15K+ base images)—no overfitting risk.
- **TAN hypernetwork has strong inductive bias**: it maps support features $\rightarrow$ affine params through a 64-dim bottleneck. With 64 hidden units, the hypernetwork has $\sim$130K params but these are *conditioned* on all K-shot support features jointly, not per-image. The bottleneck forces the network to learn a low-dimensional space of normalization transformations.
- **Identity initialization**: TAN starts as identity transform ($\gamma = 1, \beta = 0$), so early training behaves identically to vanilla GN. Deviations only occur when the gradient signal is strong enough, which acts as implicit regularization.
- **No per-class parameters**: Unlike adapters that learn separate parameters per insertion point, TAN shares the hypernetwork across all novel classes—the class-specific behavior emerges from the support set conditioning.

4 Solution 2: Dual-Pathway Gradient Decoupling (DP-GDL)

4.1 Motivation

DeFRCN's GDL uses fixed $\lambda_{rpn} = 0$ (stop gradient) and $\lambda_{rcnn} = 0.75$ (scale gradient). These values were found by grid search on COCO 10/30-shot. However:

- Optimal λ varies per class, per image, and per shot setting
- Fixed λ cannot adapt to the distributional shift between base and novel
- Different proposals within the same image may need different decoupling strengths

4.2 Detailed Implementation

4.2.1 Step 1: Learnable Gating Network (Base Training Only)

Replace fixed λ with a lightweight gating module:

Listing 4: Dynamic GDL with learned gating

```
class DynamicGDL(nn.Module):
    """Gradient Decoupled Layer with input-dependent lambda."""
    def __init__(self, channels, reduction=16):
        super().__init__()
        # Channel attention gate: maps features -> per-channel lambda
        self.gate = nn.Sequential(
            nn.AdaptiveAvgPool2d(1),
            nn.Flatten(),
            nn.Linear(channels, channels // reduction),
```

```
            nn.ReLU(inplace=True),
            nn.Linear(channels // reduction, channels),
            nn.Sigmoid()
        )
        # Initialize to match DeFRCN's fixed lambda
        # For RPN: init to ~0 (stop gradient)
        # For RCNN: init to ~0.75 (scale gradient)
        self.default_lambda = 0.75  # set per-module

    def forward(self, x):
        # Forward: apply channel-wise affine (like original GDL)
        return x  # identity in forward pass

    def backward_hook(self, grad):
        # Backward: scale gradient by learned lambda
        lambdas = self.gate(self._cached_input)  # [B, C]
        lambdas = lambdas.unsqueeze(-1).unsqueeze(-1)  # [B,C,1,1]
        return grad * lambdas
```

4.2.2 Step 2: Dual-Pathway Feature Routing

Split the backbone output into classification-relevant and localization-relevant pathways:

Listing 5: Dual-pathway feature routing

```
class DualPathwayHead(nn.Module):
    def __init__(self, in_channels=2048, roi_size=7):
        super().__init__()
        # Classification pathway: spatial average pool -> invariant
        self.cls_pool = nn.AdaptiveAvgPool2d(1)
        self.cls_proj = nn.Linear(in_channels, in_channels)

        # Localization pathway: preserve spatial info
        self.loc_proj = nn.Conv2d(in_channels, in_channels, 1)

        # Routing gate: learns to split features
        self.route_gate = nn.Sequential(
            nn.AdaptiveAvgPool2d(1),
            nn.Flatten(),
            nn.Linear(in_channels, 1),
            nn.Sigmoid()
        )

    def forward(self, roi_features):
        gate = self.route_gate(roi_features)  # [B, 1]
        # Classification: translation-invariant features
        cls_feat = self.cls_pool(roi_features).flatten(1)
        cls_feat = self.cls_proj(cls_feat)
        # Localization: translation-covariant features
        loc_feat = self.loc_proj(roi_features)
        return cls_feat, loc_feat, gate
```

4.2.3 Step 3: Orthogonality Regularization

Encourage the two pathways to learn complementary features:

Listing 6: Orthogonality loss

```python
def orthogonality_loss(cls_grad, loc_grad, eps=1e-8):
    """Penalize alignment between classification and
    localization gradient directions."""
    cls_flat = cls_grad.flatten(1)
    loc_flat = loc_grad.flatten(1)
    # Cosine similarity between gradient directions
    cos_sim = F.cosine_similarity(cls_flat, loc_flat, dim=1)
    return cos_sim.pow(2).mean()  # minimize squared cosine
```

4.2.4 Step 4: Integration

1. **Base training:** Train the gating network and dual pathways with the orthogonality loss. Total loss: $\mathcal{L} = \mathcal{L}_{rpn} + \mathcal{L}_{rcnn}^{cls} + \mathcal{L}_{rcnn}^{reg} + \lambda_{orth} \| \cos(\nabla_{cls}, \nabla_{loc}) \|^2$, with $\lambda_{orth} = 0.1$.
2. **Novel fine-tuning: Freeze** the gating network and routing module. Only train the classifier head. The learned routing persists from base training.
3. **Inference:** No changes needed—the dual pathway is transparent.

4.3 Why This Avoids Overfitting

- **All learnable components are trained during base training** on 15K+ images with 15 classes. Zero additional parameters are trained during novel fine-tuning.
- **The gating network is frozen during novel fine-tuning**—it cannot overfit to the K-shot support set. It simply applies the routing policy learned from abundant base data.
- **The routing generalizes across classes** because it operates on feature statistics (channel means via GAP), not class-specific patterns. A "small object in cluttered scene" activates the same routing whether the object is a base class (car) or novel class (bird).
- **Orthogonality regularization acts as explicit anti-overfitting**: by forcing classification and localization features to be orthogonal, it prevents the classifier from exploiting localization shortcuts that only work for the few training examples.
- **Reduced hyperparameter sensitivity**: instead of manually tuning λ_{rpn} and λ_{rcnn}, the model learns them. Fewer hyperparameters = less risk of overfitting to validation set choices.

5 Solution 3: PCB with Foundation Model Alignment (PCB-FMA)

5.1 Motivation

PCB uses a *separately pretrained* ImageNet ResNet-101 as the feature extractor for prototype construction. This is limiting because:

- ImageNet ResNet-101 was trained for 1000-class classification—its features are optimized for ImageNet categories, not VOC/COCO detection categories
- The cosine similarity in PCB operates in a space not aligned with detection features
- Foundation models (DINOv2, SigLIP) trained on billions of images with self-supervised objectives produce far more generalizable features

5.2 Detailed Implementation

5.2.1 Step 1: Replace ImageNet ResNet-101 with Frozen Foundation Model

Listing 7: Foundation model feature extraction for PCB

```python
import torch
```

```python
import torch.nn as nn

class FoundationModelPCB(nn.Module):
    """PCB with DINOv2 as the prototype feature extractor."""
    def __init__(self, model_name='dinov2_vitl14', proj_dim=1024):
        super().__init__()
        # Load frozen foundation model
        self.fm = torch.hub.load('facebookresearch/dinov2',
                                 model_name)
        self.fm.eval()
        for p in self.fm.parameters():
            p.requires_grad = False

        self.fm_dim = self.fm.embed_dim  # e.g., 1024 for ViT-L
        self.proj_dim = proj_dim

        # Lightweight cross-space projection (trained on base)
        # Maps detection RoI features to FM space
        self.det_to_fm = nn.Sequential(
            nn.Linear(2048, proj_dim),
            nn.LayerNorm(proj_dim),
            nn.GELU(),
            nn.Linear(proj_dim, proj_dim),
            nn.LayerNorm(proj_dim),
        )

        # FM RoI feature extraction
        self.fm_roi_pool = ROIPooler(
            output_size=(14, 14),  # ViT patch size
            scales=(1/14,), sampling_ratio=0,
            pooler_type="ROIAlignV2"
        )

    def extract_fm_features(self, images, boxes):
        """Extract foundation model features for RoIs."""
        with torch.no_grad():
            # Get FM patch tokens
            fm_features = self.fm.forward_features(images)
            patch_tokens = fm_features['x_norm_patchtokens']
            # Reshape to spatial feature map
            B, N, D = patch_tokens.shape
            h = w = int(N ** 0.5)
            fm_map = patch_tokens.reshape(B, h, w, D)
            fm_map = fm_map.permute(0, 3, 1, 2)  # B,D,h,w
        # Pool RoIs from FM feature map
        roi_feats = self.fm_roi_pool([fm_map], boxes)
        roi_feats = roi_feats.mean(dim=[2, 3])  # global avg pool
        return roi_feats  # [num_rois, fm_dim]
```

5.2.2 Step 2: Cross-Space Projection Training (Base Stage)

Train the projection layer $\pi : \mathbb{R}^{d_{det}} \to \mathbb{R}^{d_{fm}}$ using base-class data:

Listing 8: Cross-space projection training

```python
class CrossSpaceProjectionLoss(nn.Module):
    def __init__(self, temperature=0.07):
        super().__init__()
```

```
        self.temp = temperature

    def forward(self, det_projected, fm_features, labels):
        """
        det_projected: [N, D] - detector features projected to FM space
        fm_features: [N, D] - foundation model features for same RoIs
        labels: [N] - class labels
        """
        # Normalize
        det_norm = F.normalize(det_projected, dim=1)
        fm_norm = F.normalize(fm_features, dim=1)

        # Contrastive alignment: same-RoI features should match
        pos_sim = (det_norm * fm_norm).sum(dim=1) / self.temp
        # Cross-RoI negatives
        sim_matrix = det_norm @ fm_norm.t() / self.temp
        # InfoNCE loss
        loss = -pos_sim + torch.logsumexp(sim_matrix, dim=1)
        return loss.mean()
```

5.2.3 Step 3: Tri-Modal Score Fusion

Calibrate detection scores using three complementary signals:

Listing 9: Tri-modal score calibration

```
class TriModalCalibration:
    def __init__(self, alpha=0.4, beta=0.4, gamma=0.2):
        self.alpha = alpha  # detector score weight
        self.beta = beta    # FM visual similarity weight
        self.gamma = gamma  # FM text similarity weight (optional)
        self.text_prototypes = None  # pre-computed

    def build_text_prototypes(self, class_names, fm_model):
        """Use CLIP/SigLIP text encoder for zero-shot prototypes."""
        import clip
        texts = [f"a␣photo␣of␣a␣{name}" for name in class_names]
        text_tokens = clip.tokenize(texts)
        with torch.no_grad():
            self.text_prototypes = fm_model.encode_text(text_tokens)
            self.text_prototypes = F.normalize(
                self.text_prototypes, dim=1)

    def calibrate(self, det_score, det_roi_feat, fm_roi_feat,
                  fm_prototypes, class_id):
        """Fuse detector, FM-visual, and FM-text scores."""
        # Visual prototype similarity (in FM space)
        fm_feat_norm = F.normalize(fm_roi_feat, dim=0)
        proto_norm = F.normalize(fm_prototypes[class_id], dim=0)
        s_fm_visual = (fm_feat_norm * proto_norm).sum()

        # Text prototype similarity (optional)
        s_fm_text = 0.0
        if self.text_prototypes is not None:
            text_proto = self.text_prototypes[class_id]
            s_fm_text = (fm_feat_norm * text_proto).sum()

        # Tri-modal fusion
```

```
        s_final = (self.alpha * det_score +
                   self.beta * s_fm_visual +
                   self.gamma * s_fm_text)
        return s_final
```

5.2.4 Step 4: Full Pipeline Integration

1. **Base training:** Train DeFRCN normally + train cross-space projection π using base-class RoI pairs (detector features, FM features) with contrastive loss. Only π is new (∼4M params, trained on 15K+ base images).
2. **Novel fine-tuning:** Standard DeFRCN novel fine-tuning. π is frozen.
3. **Inference:** For each detection, extract FM features for the RoI, compute tri-modal calibration score. FM is frozen, π is frozen—**zero additional parameters trained on novel data**.

5.3 Why This Avoids Overfitting

- **Zero novel-stage learnable parameters**: The foundation model is frozen. The projection π is trained on base classes and frozen. Novel fine-tuning is identical to vanilla DeFRCN.
- **This is strictly an inference-time modification**: The calibration replaces the existing PCB's ImageNet ResNet-101 with a foundation model. No training changes means no overfitting opportunity.
- **Foundation models are inherently anti-overfitting**: DINOv2/SigLIP are trained on billions of diverse images. Their feature space is vastly more generalizable than ImageNet ResNet-101. A novel class like "sofa" (which gets 5.86 AP_{50} in our vanilla results) will have a much better prototype in DINOv2 space because DINOv2 has seen millions of sofa-like objects during pre-training.
- **Text prototypes provide zero-shot regularization**: Even without any visual support, the text prototype "a photo of a sofa" provides a reasonable prior. This acts as a Bayesian prior that prevents the prototype from collapsing to a single bad example.
- **Tri-modal fusion diversifies the signal**: Instead of relying solely on detector scores (which overfit to training examples), the FM-visual and FM-text scores provide independent, non-overfitted estimates.

5.4 Expected Gain Analysis

Table 2: Why PCB-FMA should work: per-class analysis for Split 1 1-shot

Class	Our AP_{50}	Issue	Why FM Helps
bird	54.30	Moderate	FM's fine-grained feature resolves bird vs. bg
bus	70.93	OK	Already strong; FM maintains
cow	51.15	Moderate	FM distinguishes cow from horse/sheep better
motorbike	70.17	OK	Already strong
sofa	5.86	Critical	ImageNet ResNet has poor sofa representation; DINOv2 sees millions of indoor scenes

The sofa class alone at 5.86 AP_{50} drags the average down by ∼9 points. A foundation model should dramatically improve this class due to its exposure to diverse indoor imagery

during pretraining.

6 Solution 4: Meta-Calibration (Meta-PCB)

6.1 Motivation

The PCB calibration formula $s_i^\dagger = \alpha \cdot s_i + (1-\alpha) \cdot s_i^{cos}$ is a *fixed linear function* with a single hyperparameter α. This is suboptimal because:

- The optimal calibration is likely *non-linear* (high-confidence detections need less calibration than borderline ones)
- The optimal α varies per class, per score range, and per support set quality
- Hand-tuning α on the validation set risks overfitting the hyperparameter itself

6.2 Detailed Implementation

6.2.1 Step 1: Calibration Network Architecture

Listing 10: Meta-PCB calibration network

```
class MetaCalibrationNet(nn.Module):
    """Learned non-linear calibration function.
    Replaces: s_final = alpha * s_det + (1-alpha) * s_cos
    With: s_final = C_theta(s_det, s_cos, feat_stats)
    """
    def __init__(self, feat_dim=2048, hidden=64):
        super().__init__()
        # Input: [det_score, cos_sim, support_stats, roi_stats]
        # support_stats: mean/std of support features (4 dims)
        # roi_stats: feature norm, entropy of score dist (2 dims)
        input_dim = 2 + 4 + 2  # = 8 scalar inputs

        self.calibrator = nn.Sequential(
            nn.Linear(input_dim, hidden),
            nn.ReLU(inplace=True),
            nn.Linear(hidden, hidden),
            nn.ReLU(inplace=True),
            nn.Linear(hidden, 1),
            nn.Sigmoid()  # output calibrated score in [0, 1]
        )

        # Residual connection: initialize to approximate
        # the original linear calibration
        self._init_to_linear_baseline()

    def _init_to_linear_baseline(self):
        """Init so output approx = 0.5*s_det + 0.5*s_cos"""
        # Zero-init last layer for residual learning
        nn.init.zeros_(self.calibrator[-2].weight)
        nn.init.zeros_(self.calibrator[-2].bias)

    def forward(self, det_score, cos_sim,
                support_mean, support_std,
                roi_feat_norm, score_entropy):
        x = torch.stack([
            det_score, cos_sim,
            support_mean, support_std,
            support_mean.norm(), support_std.norm(),
```

```
            roi_feat_norm, score_entropy
        ], dim=-1)
        # Residual: learned correction on top of linear baseline
        linear_baseline = 0.5 * det_score + 0.5 * cos_sim
        correction = self.calibrator(x).squeeze(-1) - 0.5
        return (linear_baseline + 0.1 * correction).clamp(0, 1)
```

6.2.2 Step 2: Episodic Meta-Training

Train the calibration network on simulated few-shot episodes constructed from base classes:

Listing 11: Meta-training loop for calibration network

```
def meta_train_calibrator(calibrator, base_detector,
                          base_dataset, num_episodes=10000,
                          n_way=5, k_shot=1):
    optimizer = torch.optim.Adam(calibrator.parameters(), lr=1e-3)

    for episode in range(num_episodes):
        # Sample a pseudo-few-shot task from base classes
        # Select n_way classes randomly from 15 base classes
        classes = random.sample(range(15), n_way)

        # Sample k_shot support images per class
        support = sample_support(base_dataset, classes, k_shot)

        # Build prototypes from support (same as PCB)
        prototypes = build_prototypes(support, imagenet_model)

        # Run detector on query images
        query_images = sample_query(base_dataset, classes, n=50)
        detections = base_detector(query_images)

        # For each detection, compute calibrated score
        losses = []
        for det in detections:
            det_score = det['score']
            cos_sim = cosine_similarity(det['roi_feat'],
                                        prototypes[det['pred_cls']])
            # Compute support statistics
            s_mean = prototypes[det['pred_cls']].mean()
            s_std = prototypes[det['pred_cls']].std()
            roi_norm = det['roi_feat'].norm()
            s_entropy = compute_score_entropy(det['all_scores'])

            calibrated = calibrator(det_score, cos_sim,
                                    s_mean, s_std,
                                    roi_norm, s_entropy)
            # Supervise with IoU-based soft label
            soft_label = compute_iou_label(det['box'],
                                           det['gt_box'])
            losses.append(F.binary_cross_entropy(
                calibrated, soft_label))

        loss = torch.stack(losses).mean()
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

6.2.3 Step 3: Integration

1. **Base training:** Train DeFRCN normally. Then meta-train calibration network using base-class episodes (separate from DeFRCN training, using frozen base detector).
2. **Novel fine-tuning:** Standard DeFRCN. Calibration network is frozen.
3. **Inference:** Replace $s^\dagger = \alpha \cdot s + (1-\alpha) \cdot s^{cos}$ with $s^\dagger = \mathcal{C}_\theta(s, s^{cos}, \text{stats})$.

6.3 Why This Avoids Overfitting

- **The calibration network is trained on base classes and frozen during novel evaluation**. It *never sees* novel class data during training. Zero risk of novel-class overfitting.
- **Episodic training simulates the exact few-shot condition**: by constructing pseudo-few-shot tasks from base classes, the calibrator learns to generalize to *any* novel class with K support examples. This is the core insight of meta-learning—learn the learning algorithm itself.
- **Tiny architecture with residual connection**: The calibrator has only $8 \times 64 + 64 \times 64 + 64 \times 1 = 4,672$ parameters. The residual connection ensures it starts as vanilla PCB and only learns corrections. The small capacity acts as a regularizer.
- **Input features are class-agnostic statistics**: The calibrator operates on scalar inputs (scores, similarity, norms, entropy)—not on raw high-dimensional features. This means it cannot memorize class-specific patterns; it learns a *general calibration policy* based on prediction confidence signals.
- **10,000 episodes $\gg$ 4,672 parameters**: The meta-training data is effectively infinite (random episode sampling from base classes), so the calibrator cannot overfit to specific episodes.

7 Solution 5: Uncertainty-Guided Prototype Refinement (UPR-TTA)

7.1 Motivation

The transductive PCB approach (Section 9 of our modified calibration layer) collects pseudo-labels to refine prototypes. However, it uses *ad-hoc thresholds* (`TRANS_MIN_SCORE=0.70`, `TRANS_MIN_SIM`) to decide which detections to trust. This is fragile: too strict $\rightarrow$ too few pseudo-labels (only 25 collected from 4952 images in our run); too loose $\rightarrow$ false positives corrupt prototypes.

The solution: use *epistemic uncertainty* to make principled decisions about which detections to trust.

7.2 Detailed Implementation

7.2.1 Step 1: MC-Dropout Uncertainty Estimation

Enable dropout in the Res5 ROI head during inference and perform multiple stochastic forward passes:

Listing 12: MC-Dropout uncertainty estimation

```
class MCDropoutUncertainty:
    def __init__(self, model, num_passes=10, dropout_rate=0.1):
        self.model = model
        self.T = num_passes
        self.dropout_rate = dropout_rate

    def enable_mc_dropout(self):
```

```
        """Enable dropout in res5 head for MC estimation."""
        for module in self.model.roi_heads.res5.modules():
            if isinstance(module, nn.Dropout):
                module.train()  # keep dropout active
            # Also enable dropout for conv layers via
            # spatial dropout
        # Add dropout after res5 if not present
        if not hasattr(self.model.roi_heads, 'mc_dropout'):
            self.model.roi_heads.mc_dropout = nn.Dropout2d(
                p=self.dropout_rate)

    @torch.no_grad()
    def predict_with_uncertainty(self, images, proposals):
        """Run T forward passes, compute mean + variance."""
        self.enable_mc_dropout()
        all_scores = []
        all_features = []

        for t in range(self.T):
            # Stochastic forward pass
            features = self.model.backbone(images.tensor)
            roi_features = self.model.roi_heads._shared_roi_transform(
                [features['res4']], proposals)
            # Apply MC dropout
            roi_features = self.model.roi_heads.mc_dropout(
                roi_features)
            roi_features = roi_features.mean(dim=[2, 3])

            scores = self.model.roi_heads.box_predictor(
                roi_features)[0]
            scores = F.softmax(scores, dim=1)

            all_scores.append(scores)
            all_features.append(roi_features)

        scores_stack = torch.stack(all_scores, dim=0)  # [T,N,C]
        feats_stack = torch.stack(all_features, dim=0)  # [T,N,D]

        # Predictive mean and uncertainty
        mean_scores = scores_stack.mean(dim=0)      # [N, C]
        epistemic_var = scores_stack.var(dim=0)      # [N, C]
        mean_features = feats_stack.mean(dim=0)      # [N, D]
        feat_var = feats_stack.var(dim=0).mean(dim=1)  # [N]

        return {
            'mean_scores': mean_scores,
            'epistemic_var': epistemic_var,
            'mean_features': mean_features,
            'feat_uncertainty': feat_var,
        }
```

7.2.2 Step 2: Uncertainty-Guided Pseudo-Label Selection

Listing 13: Principled pseudo-label selection

```
class UncertaintyGuidedPseudoLabeler:
    def __init__(self, score_thresh=0.5, uncertainty_thresh=0.05,
```

```
                 max_per_class=10):
        self.score_thresh = score_thresh
        self.unc_thresh = uncertainty_thresh
        self.max_per_class = max_per_class

    def select_pseudo_labels(self, predictions, uncertainties):
        """Select high-confidence, low-uncertainty detections
        as pseudo-support."""
        pseudo_labels = {}
        scores = predictions['mean_scores']
        unc = predictions['epistemic_var']
        features = predictions['mean_features']

        for i in range(len(scores)):
            cls = scores[i].argmax().item()
            cls_score = scores[i, cls].item()
            cls_unc = unc[i, cls].item()
            feat_unc = predictions['feat_uncertainty'][i].item()

            # Key insight: select iff HIGH score AND LOW uncertainty
            # High score + high uncertainty = model is guessing
            # High score + low uncertainty = model is confident
            if (cls_score >= self.score_thresh and
                cls_unc <= self.unc_thresh):

                # Weight by inverse uncertainty
                quality = cls_score / (cls_unc + 1e-8)

                if cls not in pseudo_labels:
                    pseudo_labels[cls] = []
                pseudo_labels[cls].append({
                    'feature': features[i],
                    'score': cls_score,
                    'uncertainty': cls_unc,
                    'quality': quality,
                })

        # Keep top-k per class by quality
        for cls in pseudo_labels:
            pseudo_labels[cls].sort(
                key=lambda x: x['quality'], reverse=True)
            pseudo_labels[cls] = (
                pseudo_labels[cls][:self.max_per_class])

        return pseudo_labels
```

7.2.3 Step 3: Uncertainty-Adaptive PCB Alpha

Listing 14: Adaptive alpha based on prediction uncertainty

```
def uncertainty_adaptive_alpha(det_score, cos_sim,
                               epistemic_unc, base_alpha=0.5):
    """High uncertainty -> rely more on PCB (prototype).
    Low uncertainty -> rely more on detector."""
    # Normalize uncertainty to [0, 1] range
    unc_normalized = min(epistemic_unc / 0.1, 1.0)
```

```
    # High uncertainty: decrease alpha (more PCB influence)
    # Low uncertainty: increase alpha (trust detector)
    alpha = base_alpha + (1.0 - base_alpha) * (1.0 - unc_normalized)
    alpha = max(0.1, min(0.9, alpha))

    return alpha * det_score + (1 - alpha) * cos_sim
```

7.2.4 Step 4: One-Step Test-Time Adaptation

Listing 15: Test-time adaptation of classifier head

```
class TestTimeAdaptation:
    def __init__(self, model, lr=0.001, entropy_thresh=0.3):
        self.model = model
        self.lr = lr
        self.entropy_thresh = entropy_thresh
        # Only adapt the final classifier head
        self.params = [p for n, p in model.named_parameters()
                       if 'box_predictor.cls_score' in n]
        self.optimizer = torch.optim.SGD(self.params, lr=lr)
        # Save initial state for reset
        self.initial_state = {n: p.clone()
                              for n, p in model.named_parameters()
                              if 'box_predictor.cls_score' in n}

    def adapt_step(self, features, pseudo_labels):
        """One gradient step using pseudo-labeled detections."""
        if not pseudo_labels:
            return

        # Collect pseudo-labeled features and targets
        feats, targets = [], []
        for cls, items in pseudo_labels.items():
            for item in items:
                if item['uncertainty'] < self.entropy_thresh:
                    feats.append(item['feature'])
                    targets.append(cls)

        if len(feats) < 3:  # need minimum samples
            return

        feats = torch.stack(feats)
        targets = torch.tensor(targets, device=feats.device)

        # One-step update with entropy minimization
        self.optimizer.zero_grad()
        logits = self.model.roi_heads.box_predictor.cls_score(
            feats)
        loss = F.cross_entropy(logits, targets)
        loss.backward()
        self.optimizer.step()

    def reset(self):
        """Reset to initial weights after each test image."""
        for n, p in self.model.named_parameters():
            if n in self.initial_state:
                p.data.copy_(self.initial_state[n])
```

7.2.5 Step 5: Full Pipeline

1. **Base training:** Standard DeFRCN + add dropout layers to Res5 head (train with dropout for regularization benefit).
2. **Novel fine-tuning:** Standard DeFRCN (dropout acts as regularizer—actual benefit for K-shot training).
3. **Inference:** Two-pass protocol:
   - *Pass 1*: Run MC-Dropout on all test images. Collect pseudo-labels using uncertainty-guided selection. Rebuild prototypes.
   - *Pass 2*: Run final evaluation with rebuilt prototypes and uncertainty-adaptive α. Optionally apply one-step TTA per image.

7.3 Why This Avoids Overfitting

- **MC-Dropout is parameter-free at test time**: Uncertainty estimation requires zero additional parameters. It simply runs the same model multiple times with different dropout masks.
- **Dropout during novel training is actually *beneficial***: Adding dropout to the Res5 head during the K-shot fine-tuning provides regularization that *reduces* overfitting. This is one of the most established anti-overfitting techniques.
- **Uncertainty-based selection is principled and self-correcting**: Unlike fixed thresholds (`MIN_SCORE=0.7`), the uncertainty criterion adapts to the model's actual confidence. A well-calibrated model will naturally select only reliable pseudo-labels. If the model is uncertain about everything, no pseudo-labels are selected—safely falling back to vanilla PCB.
- **TTA with reset prevents accumulation of errors**: The classifier is adapted for one test image, then *reset to initial weights*. No errors accumulate across test images. The one-step update with 3+ pseudo-labeled features and a learning rate of 0.001 produces a minimal weight change that cannot cause catastrophic forgetting.
- **Bayesian justification**: MC-Dropout approximates Bayesian inference over the posterior $p(\theta|D_{K\text{-shot}})$. The uncertainty estimate captures *epistemic uncertainty*—what the model doesn't know due to limited data. This is exactly the right quantity to use for deciding when to trust predictions in a data-scarce regime.

7.4 Comparison with Current Transductive Approach

Table 3: Current transductive vs. UPR-TTA

Property	**Current Transductive**	**UPR-TTA**
Selection criterion	Fixed score threshold	Uncertainty-based
Threshold	Manually tuned (0.70)	Adaptive to model confidence
Pseudo-labels collected (1-shot)	25 from 4952 images	Expected 100–500
False positive risk	High (high-score FPs pass)	Low (high-unc FPs filtered)
Prototype corruption	Possible	Mitigated by uncertainty
Additional parameters	Zero	Zero (dropout is free)
Novel training benefit	None	Dropout regularization

8 Comparative Summary of All Solutions

Key insight across all solutions: Every proposal either (a) adds parameters only during base training (15K+ images), or (b) modifies only the inference pipeline. **Zero solutions**

Table 4: Summary comparison: overfitting risk, novel-stage parameters, and expected gains

Solution	Novel-stage new params	Overfit risk	Expected gain	Impl. effort	Venue target
BA-FSD (Norm)	∼130K (TAN)	Low	+2–4 AP	Medium	CVPR
DP-GDL (Routing)	0 (frozen)	None	+1–3 AP	High	CVPR
PCB-FMA (Foundation)	0 (frozen)	None	+3–8 AP	Low	ICLR
Meta-PCB (Meta-Cal)	0 (frozen)	None	+2–5 AP	Medium	NeurIPS
UPR-TTA (Uncertainty)	0 (dropout)	None	+2–4 AP	Low	ICLR

add learnable parameters during the K-shot novel fine-tuning stage. This is the fundamental design principle that prevents overfitting in FSOD.

8.1 Recommended Priority

1. **PCB-FMA** (Solution 3): Highest expected gain, lowest implementation effort, timely topic. Start here.
2. **UPR-TTA** (Solution 5): Complements PCB-FMA naturally (uncertainty-guided prototype refinement *in* the foundation model's feature space). Combine with Solution 3 for a complete paper.
3. **Meta-PCB** (Solution 4): Strong theoretical contribution but higher implementation effort. Good standalone paper.
4. **DP-GDL** (Solution 2): Requires re-training the base model with modified architecture. Higher risk but more novel.
5. **BA-FSD** (Solution 1): Important for practical reproducibility but harder to sell as a standalone contribution. Best as a component of a larger paper.

8.2 Combined Paper Architecture (Strongest Submission)

The strongest A*-level paper combines Solutions 3 + 5:

Title: "*Foundation-Aligned Prototypical Calibration with Uncertainty-Guided Refinement for Few-Shot Object Detection*"

Contributions:

1. Replace PCB's ImageNet ResNet-101 with DINOv2 + learned cross-space projection (trained on base classes)
2. Add text-conditioned prototypes as zero-shot prior
3. Use MC-Dropout uncertainty to guide transductive prototype refinement in the foundation model's feature space
4. Comprehensive experiments: VOC (3 splits × 5 shots), COCO (5 shots), cross-domain (COCO→VOC)

Why this combination is synergistic:

- Foundation model features provide *better prototypes* → better initial calibration
- Better initial calibration → higher-quality pseudo-labels for transductive refinement
- Uncertainty-guided selection in FM space is more reliable than in ImageNet space (FM has better calibrated features)
- Each component is independently ablatable (clean ablation study)